Chapter 9

Transaction Management and Concurrency Control

What is a Transaction?

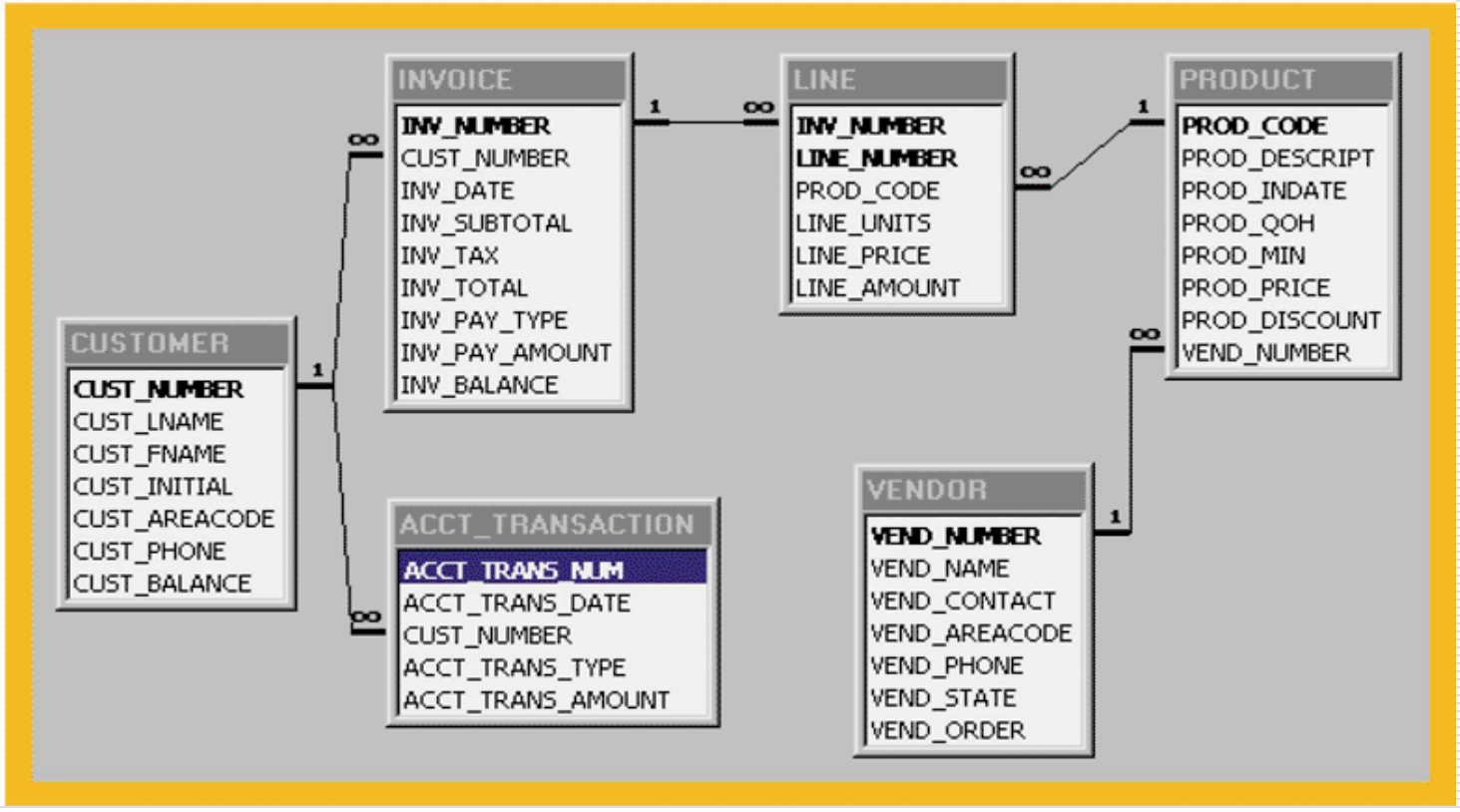
- Any action that reads from and/or writes to a database may consist of
  - Simple SELECT statement to generate a list of table contents
  - A series of related UPDATE statements to change the values of attributes in various tables
  - A series of INSERT statements to add rows to one or more tables
  - A combination of SELECT, UPDATE, and INSERT statements

What is a Transaction?

- A *logical* unit of work that must be either entirely completed or aborted
- Successful transaction changes the database from one *consistent* state to another
  - One in which all data integrity constraints are satisfied
- Most real-world database transactions are formed by two or more database requests
  - The equivalent of a single SQL statement in an application program or transaction

The Relational Schema for the Ch09_SaleCo Database

FIGURE 9.1 THE RELATIONAL SCHEMA FOR THE CH09_SALECO DATABASE



Evaluating Transaction Results

- Not all transactions update the database
- SQL code represents a transaction because database was accessed
- Improper or incomplete transactions can have a devastating effect on database integrity
  - Some DBMSs provide means by which user can define enforceable constraints based on business rules
  - Other integrity rules are enforced automatically by the DBMS when table structures are properly defined, thereby letting the DBMS validate some transactions

Transaction Processing

- For example, a transaction may involve
  - The creation of a new invoice
  - Insertion of an row in the LINE table
  - Decreasing the quantity on hand by 1
  - Updating the customer balance
  - Creating a new account transaction row
- If the system fails between the first and last step, the database will no longer be in a consistent state

Figure 8.2

Transaction Properties

- Atomicity
  - Requires that *all* operations (SQL requests) of a transaction be completed; if not, then the transaction is aborted
  - A transaction is treated as a single, indivisible, logical unit of work
- Durability
  - Indicates permanence of database's consistent state
  - When a transaction is complete, the database reaches a consistent state. That state can not be lost even if the system fails

Transaction Properties

- Serializability
  - Ensures that the concurrent execution of several transactions yields consistent results
  - Multiple, concurrent transactions appear as if they executed in serial order (one after another)
- Isolation
  - Data used during execution of a transaction cannot be used by second transaction until first one is completed

Transaction Management with SQL

- ANSI has defined standards that govern SQL database transactions
- Transaction support is provided by two SQL statements: COMMIT and ROLLBACK
- ANSI standards require that, when a transaction sequence is initiated by a user or an application program,it must continue through all succeeding SQL statements until one of four events occurs

Transaction Management with SQL

1. A COMMIT statement is reached- all changes are permanently recorded within the database
2. A ROLLBACK is reached – all changes are aborted and the database is restored to a previous consistent state
3. The end of the program is successfully reached – equivalent to a COMMIT
4. The program abnormally terminates and a rollback occurs

The Transaction Log

- Keeps track of all transactions that updatethe database. It contains:
  - A record for the beginning of transaction
  - For each transaction component (SQL statement)
    - Type of operation being performed (update, delete, insert)
    - Names of objects affected by the transaction (the name of the table)
    - "Before" and "after" values for updated fields
    - Pointers to previous and next transaction log entries for the same transaction
  - The ending (COMMIT) of the transaction
- Increases processing overhead but the ability to restore a corrupted database is worth the price

The Transaction Log

- Increases processing overhead but the ability to restore a corrupted database is worth the price
- If a system failure occurs, the DBMS will examine the log for all uncommitted or incomplete transactions and it will restore the database to a previous state
- The log it itself a database and to maintain its integrity many DBMSs will implement it on several different disks to reduce the risk of system failure

A Transaction Log

TABLE 9.1 A TRANSACTION LOG

TRL ID	TRX NUM	PREV PTR	NEXT PTR	OPERATION	TABLE	ROW ID	ATTRIBUTE	BEFORE VALUE	AFTER VALUE
341	101	Null	352	START	****Start Transaction				
352	101	341	363	UPDATE	PRODUCT	1558-QW1	PROD_QOH	25	23
363	101	352	365	UPDATE	CUSTOMER	10011	CUST_BALANCE	525.75	615.73
365	101	363	Null	COMMIT	**** End of Transaction				

TRL_ID = Transaction log record ID

PTR = Pointer to a transaction log record ID

TRX_NUM = Transaction number

(Note: The transaction number is automatically assigned by the DBMS.)

Concurrency Control

- The coordination of the simultaneous execution of transactions in a multiprocessing database is known as ***concurrency control***

- The objective of concurrency control is to ensure the serializability of transactions in a multiuser database environment

Concurrency Control

- Important → simultaneous execution of transactions over a shared database can create several data integrity and consistency problems
- The three main problems are:
  - lost updates
  - uncommitted data
  - inconsistent retrievals

Normal Execution of Two Transactions

TABLE 9.2 NORMAL EXECUTION OF TWO TRANSACTIONS

TIME	TRANSACTION	STEP	STORED VALUE
1	T1	Read PROD_QOH	35
2	T1	PROD_QOH = 35 + 100	
3	T1	Write PROD_QOH	135
4	T2	Read PROD_QOH	135
5	T2	PROD_QOH = 135 – 30	
6	T2	Write PROD_QOH	105

Lost Updates

TABLE 9.3 LOST UPDATES

TIME	TRANSACTION	STEP	STORED VALUE
1	T1	Read PROD_QOH	35
2	T2	Read PROD_QOH	35
3	T1	PROD_QOH = 35 + 100	
4	T2	PROD_QOH = 35 – 30	
5	T1	Write PROD_QOH **(Lost update)**	135
6	T2	Write PROD_QOH	5

Uncommited Data Problem

- Uncommitted data occurs when two transactions execute concurrently and the first is rolled back after the second has already accessed the uncommitted data
  - This violates the isolation property of transactions

Correct Execution of Two Transactions

TABLE 9.4 CORRECT EXECUTION OF TWO TRANSACTIONS

TIME	TRANSACTION	STEP	STORED VALUE
1	T1	Read PROD_QOH	35
2	T1	PROD_QOH = 35 + 100	
3	T1	Write PROD_QOH	135
4	T1	*****ROLLBACK *****	35
5	T2	Read PROD_QOH	35
6	T2	PROD_QOH = 35 – 30	
7	T2	Write PROD_QOH	5

An Uncommitted Data Problem

TABLE 9.5 AN UNCOMMITTED DATA PROBLEM

TIME	TRANSACTION	STEP	STORED VALUE
1	T1	Read PROD_QOH	35
2	T1	PROD_QOH = 35 + 100	
3	T1	Write PROD_QOH	135
4	T2	Read PROD_QOH **(Read uncommitted data)**	135
5	T2	PROD_QOH = 135 – 30	
6	T1	***** ROLLBACK *****	35
7	T2	Write PROD_QOH	105

Inconsistent Retrieval Problem

- Occur when a transaction calculates some aggregate functions over a set of data while transactions are updating the data
  - Some data may be read after they are changed and some before they are changed yielding inconsistent results

Retrieval During Update

TABLE 9.6 RETRIEVAL DURING UPDATE

TRANSACTION T1	TRANSACTION T2
SELECT SUM(PROD_QOH) FROM PRODUCT	UPDATE PRODUCT SET PROD_QOH = PROD_QOH + 10 WHERE PROD_CODE = '1546-QQ2'
	UPDATE PRODUCT SET PROD_QOH = PROD_QOH – 10 WHERE PROD_CODE = '1558-QW1'
	COMMIT;

Transaction Results: Data Entry Correction

TABLE 9.7 TRANSACTION RESULTS: DATA ENTRY CORRECTION

	BEFORE	AFTER
PROD_CODE	PROD_QOH	PROD_QOH
11QER/31	8	8
13-Q2/P2	32	32
1546-QQ2	15	(15 + 10) → 25
1558-QW1	23	(23 – 10) → 13
2232-QTY	8	8
2232-QWE	6	6
Total	**92**	**92**

Inconsistent Retrievals

TABLE 9.8 INCONSISTENT RETRIEVALS

TIME	TRANSACTION	ACTION	VALUE	TOTAL
1	T1	Read PROD_QOH for PROD_CODE = '11QER/31'	8	8
2	T1	Read PROD_QOH for PROD_CODE = '13-Q2/P2'	32	40
3	T2	Read PROD_QOH for PROD_CODE = '1546-QQ2'	15	
4	T2	PROD_QOH = 15 + 10		
5	T2	Write PROD_QOH for PROD_CODE = '1546-QQ2'	25	
6	T1	Read PROD_QOH for PROD_CODE = '1546-QQ2'	25	(After) 65
7	T1	Read PROD_QOH for PROD_CODE = '1158-QW1'	23	(Before) 88
8	T2	Read PROD_QOH for PROD_CODE = '1558-QW1'	23	
9	T2	PROD_QOH = 23 – 10		
10	T2	Write PROD_QOH for PROD_CODE = '1558-QW1'	13	
11	T2	***** COMMIT *****		
12	T1	Read PROD_QOH for PROD_CODE = '2232-QTY'	8	96
13	T1	Read PROD_QOH for PROD_CODE = '2232-QWE'	6	102

The Scheduler

- Special DBMS program: establishes order of operations within which concurrent transactions are executed
- Interleaves the execution of database operations to ensure serializability and isolation of transactions
  - To determine the appropriate order, the scheduler bases its actions on concurrency control algorithms such as locking and time stamping

The Scheduler (continued)

- Ensures computer's central processing unit (CPU) is used efficiently
  - Default would be FIFO without preemption – idle CPU (during I/O) is inefficient use of the resource and result in unacceptable response times in within the multiuser DBMS environment
- Facilitates data isolation to ensure that two transactions do not update the same data element at the same time

Read/Write Conflict Scenarios: Conflicting Database Operations Matrix

TABLE 9.9 READ/WRITE CONFLICT SCENARIOS: CONFLICTING DATABASE OPERATIONS MATRIX

	TRANSACTIONS		
	T1	T2	RESULT
Operations	Read	Read	No conflict
	Read	Write	Conflict
	Write	Read	Conflict
	Write	Write	Conflict

- Transactions T1 and T2 are executing concurrently over the same data
- When they both access the data and at least one is executing a WRITE, a conflict can occur

Concurrency Control with Locking Methods

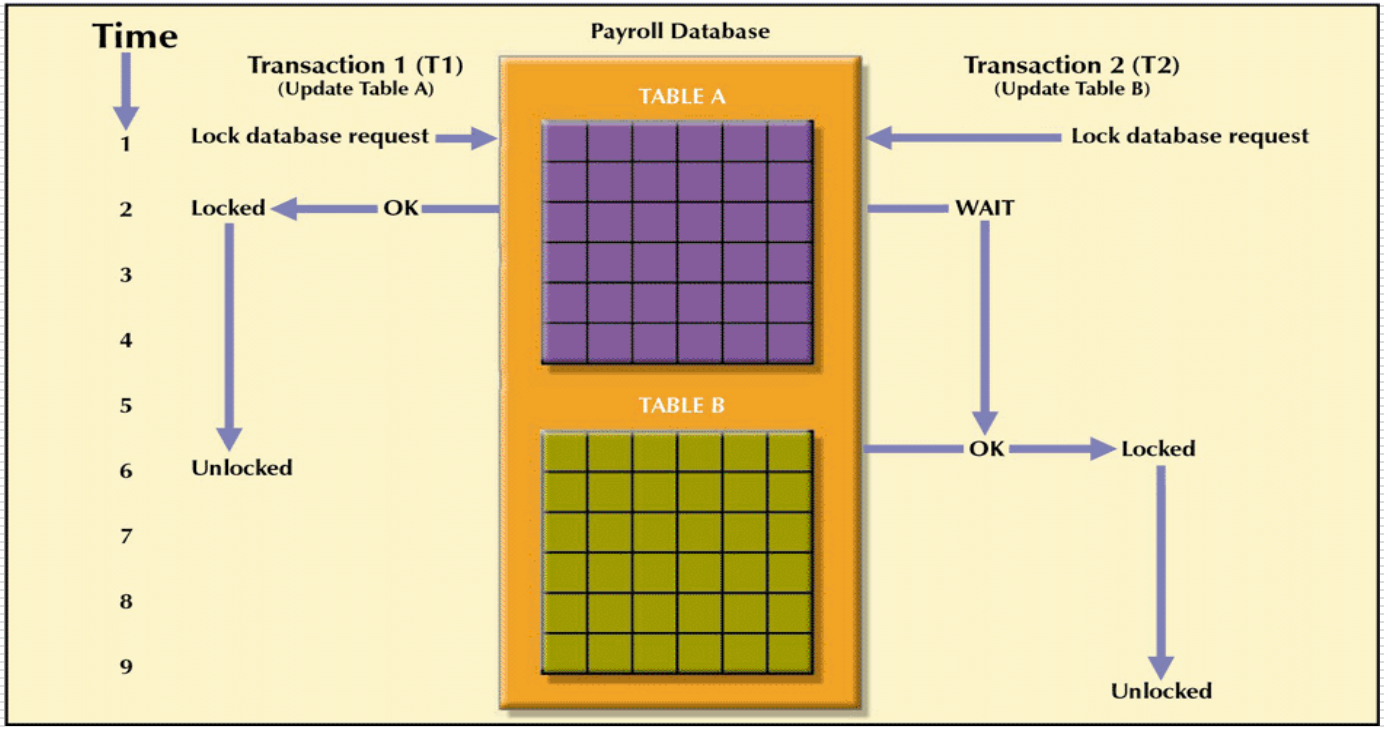
- Lock
  - Guarantees exclusive use of a data item to a current transaction
    - T2 does not have access to a data item that is currently being used by T1
    - Transaction acquires a lock prior to data access; the lock is released when the transaction is complete
  - Required to prevent another transaction from reading inconsistent data
- Lock manager
  - Responsible for assigning and policing the locks used by the transactions

Lock Granularity

- Indicates the level of lock use
- Locking can take place at the following levels:
  - Database-level lock
    - Entire database is locked
  - Table-level lock
    - Entire table is locked
  - Page-level lock
    - Entire diskpage is locked
  - Row-level lock
    - Allows concurrent transactions to access different rows of the same table, even if the rows are located on the same page
  - Field-level lock
    - Allows concurrent transactions to access the same row, as long as they require the use of different fields (attributes) within that row

A Database-Level Locking Sequence

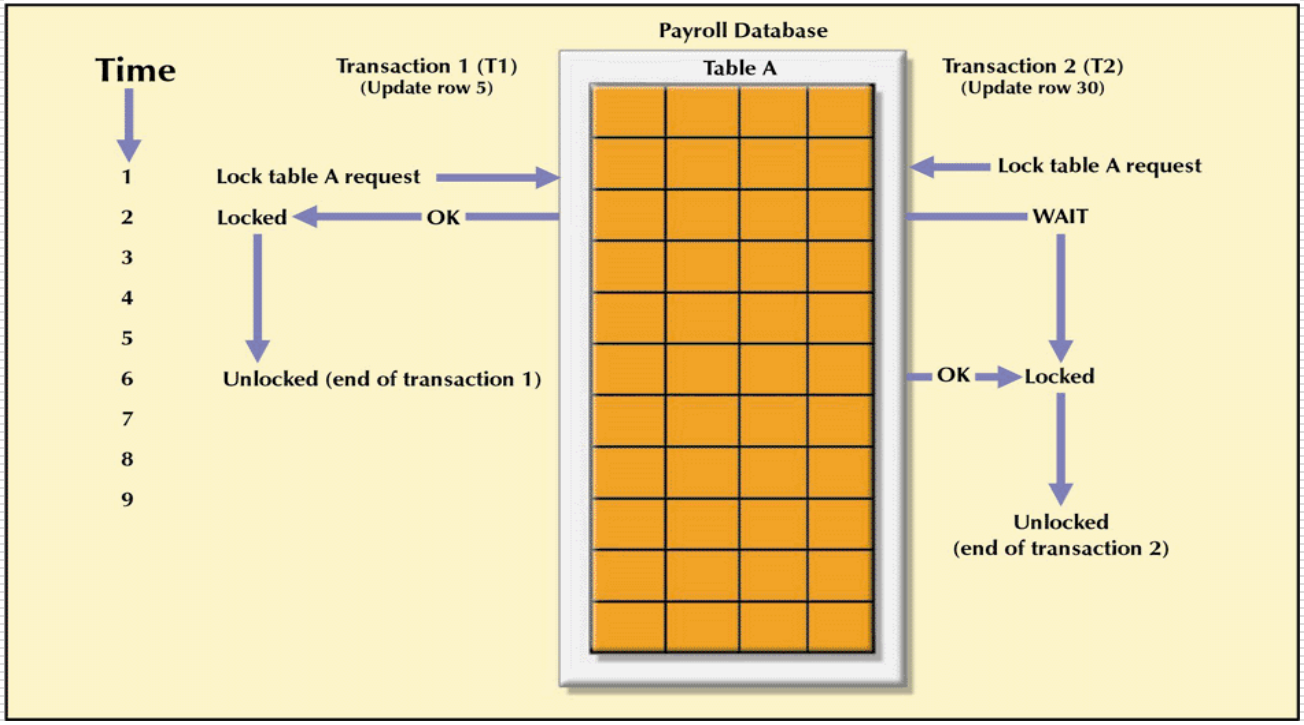
FIGURE 9.3 A DATABASE-LEVEL LOCKING SEQUENCE



- Good for batch processing but unsuitable for online multi-user DBMSs
- T1 and T2 can not access the same database concurrently even if they use different tables

Table-Level Lock

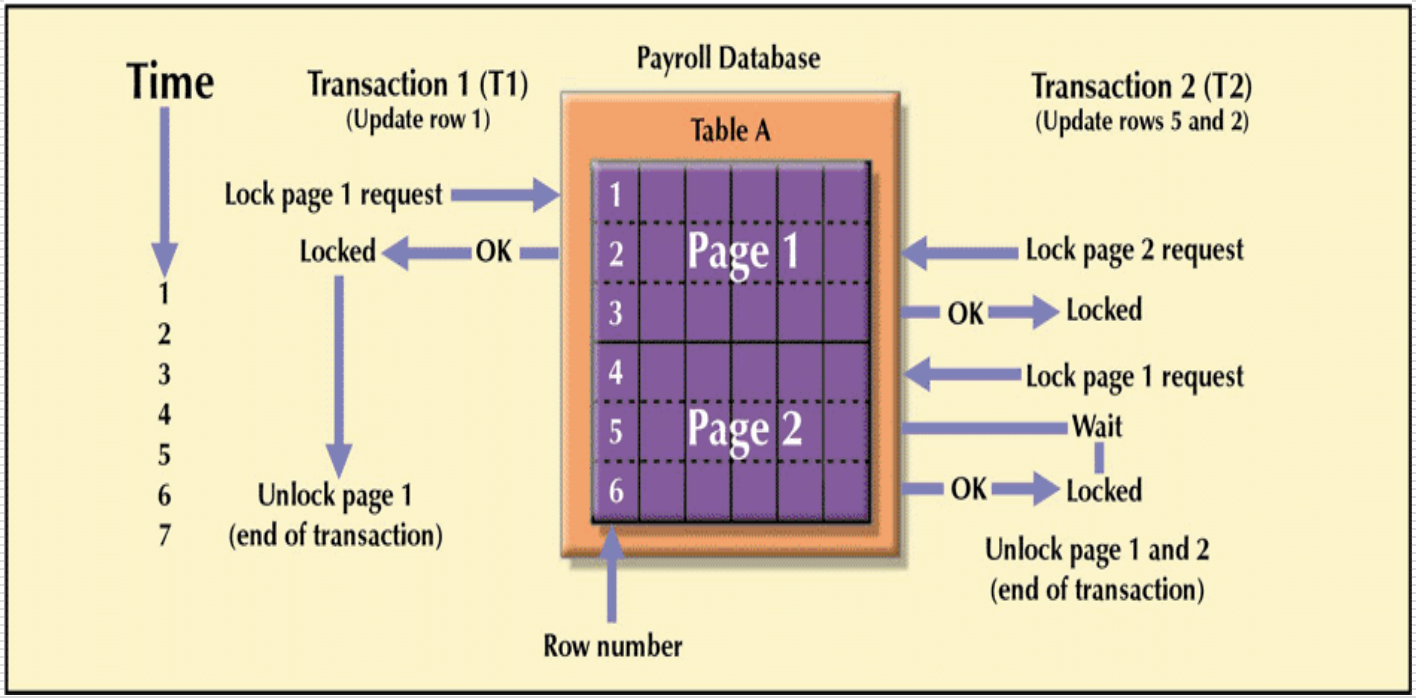
FIGURE 9.4 AN EXAMPLE OF A TABLE-LEVEL LOCK



- T1 and T2 can access the same database concurrently as long as they use different tables
- Can cause bottlenecks when many transactions are trying to access the same table (even if the transactions want to access different parts of the table and would not interfere with each other)
- Not suitable for multi-user DBMSs

Page-Level Lock

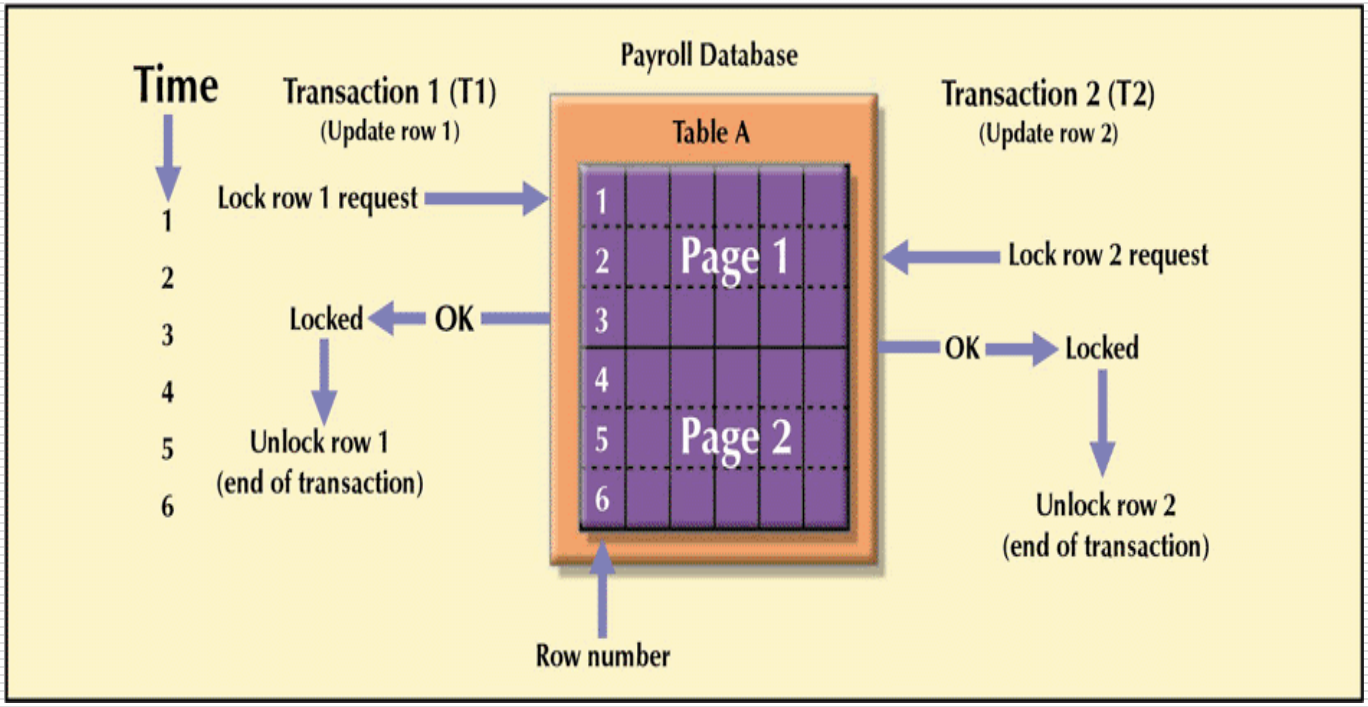
FIGURE 9.5 AN EXAMPLE OF A PAGE-LEVEL LOCK



- An entire disk page is locked (a table can span several pages and each page can contain several rows of one or more tables)
- Most frequently used multi-user DBMS locking method

Row-Level Lock

FIGURE 9.6 AN EXAMPLE OF A ROW-LEVEL LOCK



- Concurrent transactions can access different rows of the same table even if the rows are located on the same page
- Improves data availability but with high overhead (each row has a lock that must be read and written to)

Field-Level Lock

- Allows concurrent transactions to access the same row as long as they require the use of different fields with that row
- Most flexible lock buy requires an extremely high level of overhead

Binary Locks

- Has only two states: locked (1) or unlocked (0)
- Eliminates "Lost Update" problem – the lock is not released until the write statement is completed
  - Can not use PROD_QOH until it has been properly updated
- Considered too restrictive to yield optimal concurrency conditions as it locks even for two READs when no update is being done

TABLE 9.10 AN EXAMPLE OF A BINARY LOCK

TIME	TRANSACTION	STEP	STORED VALUE
1	T1	Lock PRODUCT	
2	T1	Read PROD_QOH	15
3	T1	PROD_QOH = 15 + 10	
4	T1	Write PROD_QOH	25
5	T1	Unlock PRODUCT	
6	T2	Lock PRODUCT	
7	T2	Read PROD_QOH	23
8	T2	PROD_QOH = 23 – 10	
9	T2	Write PROD_QOH	13
10	T2	Unlock PRODUCT	

Shared/Exclusive Locks

- Exclusive lock
  - Access is specifically reserved for the transaction that locked the object
  - Must be used when the potential for conflict exists – when a transaction wants to update a data item and no locks are currently held on that data item by another transaction
  - *Granted if and only if no other locks are held on the data item*
- Shared lock
  - Concurrent transactions are granted Read access on the basis of a common lock
  - Issued when a transaction wants to read data and no exclusive lock is held on that data item
    - Multiple transactions can each have a shared lock on the same data item if they are all just reading it
- Mutual Exclusive Rule
  - Only one transaction at a time can own an exclusive lock in the same object

Shared/Exclusive Locks

- Increased overhead
  - The type of lock held must be known before a lock can be granted
  - Three lock operations exist:
    - READ_LOCK to check the type of lock
    - WRITE_LOCK to issue the lock
    - UNLOCK to release the lock
  - A lock can be upgraded from share to exclusive and downgraded from exclusive to share
- Two possible major problems may occur
  - The resulting transaction schedule may not be serializable
  - The schedule may create deadlocks

Two-Phase Locking to Ensure Serializability

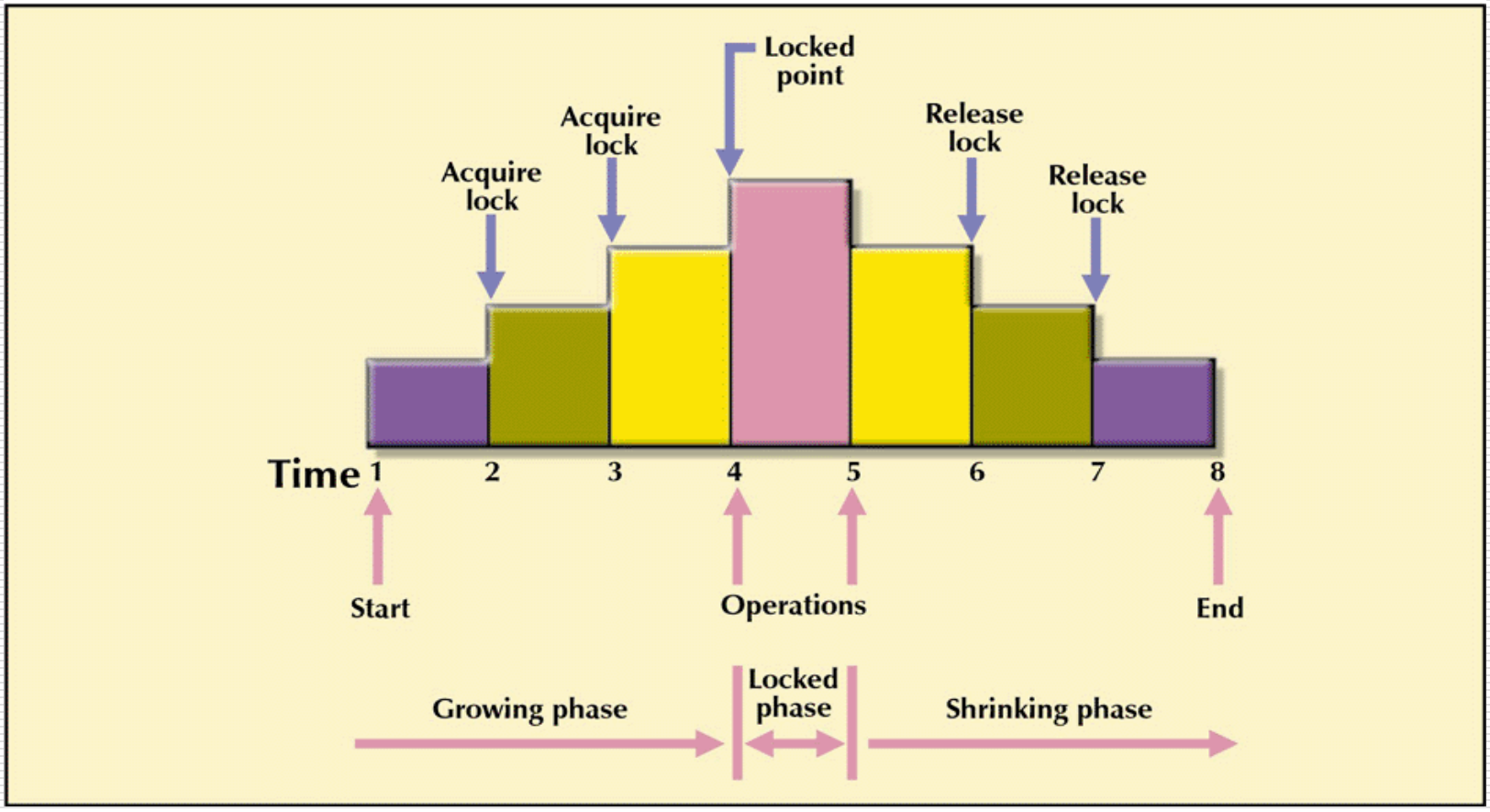
- Defines how transactions acquire and relinquish locks
- Guarantees serializability, but it does not prevent deadlocks
  - *Growing phase*, in which a transaction acquires all the required locks without unlocking any data
  - *Shrinking phase*, in which a transaction releases all locks and cannot obtain any new lock

Two-Phase Locking to Ensure Serializability

- Governed by the following rules:
  - Two transactions cannot have conflicting locks
  - No unlock operation can precede a lock operation in the same transaction
  - No data are affected until all locks are obtained—that is, until the transaction is in its locked point

Two-Phase Locking Protocol

FIGURE 9.7 TWO-PHASE LOCKING PROTOCOL



Deadlocks

- Condition that occurs when two transactions wait for each other to unlock data
  - T1 needs data items X and Y; T needs Y and X
  - Each gets the its first piece of data but then waits to get the second (which the other transaction is already holding) – ***deadly embrace***
- Possible only if one of the transactions wants to obtain an exclusive lock on a data item
  - No deadlock condition can exist among *shared* locks
- Control through
  - Prevention
  - Detection
  - Avoidance

How a Deadlock Condition Is Created

TABLE 9.11 HOW A DEADLOCK CONDITION IS CREATED

TIME	TRANSACTION	REPLY	LOCK STATUS	
0			Data X	Data Y
1	T1:LOCK(X)	OK	Unlocked	Unlocked
2	T2: LOCK(Y)	OK	Locked	Unlocked
3	T1:LOCK(Y)	WAIT	Locked	Locked
4	T2:LOCK(X)	WAIT	Locked **Deadlock**	Locked
5	T1:LOCK(Y)	WAIT	Locked	Locked
6	T2:LOCK(X)	WAIT	Locked	Locked
7	T1:LOCK(Y)	WAIT	Locked	Locked
8	T2:LOCK(X)	WAIT	Locked	Locked
9	T1:LOCK(Y)	WAIT	Locked	Locked
…	……	……	……	……
…	……	……	……	……
…	……	……	……	……
…	……	……	……	……

Concurrency Control with Time Stamping Methods

- Assigns a global unique time stamp to each transaction
  - All database operations within the same transaction must have the same time stamp
- Produces an explicit order in which transactions are submitted to the DBMS
- Uniqueness
  - Ensures that no equal time stamp values can exist
- Monotonicity
  - Ensures that time stamp values always increase
- Disadvantage
  - Each value stored in the database requires two additional time stamp fields – last read, last update

Wait/Die and Wound/Wait Schemes

- Wait/die
  - Older transaction waits and the younger is rolled back and rescheduled
- Wound/wait
  - Older transaction rolls back the younger transaction and reschedules it
- In the situation where a transaction is requests multiple locks, each lock request has an associated time-out value. If the lock is not granted before the time-out expires, the transaction is rolled back

Wait/Die and Wound/Wait Concurrency Control Schemes

TABLE 9.12 WAIT/DIE AND WOUND/WAIT CONCURRENCY CONTROL SCHEMES

TRANSACTION REQUESTING LOCK	TRANSACTION OWNING LOCK	WAIT/DIE SCHEME	WOUND/WAIT SCHEME
T1 (11548789)	T2 (19562545)	• T1 waits until T2 is completed and T2 releases its locks.	• T1 preempts (rolls back) T2. T2 is rescheduled using the same time stamp.
T2 (19562545)	T1 (11548789)	• T2 Dies (rolls back). • T2 is rescheduled using the same time stamp.	• T2 waits until T1 is completed and T1 releases its locks.

Concurrency Control with Optimistic Methods

- Optimistic approach
  - Based on the assumption that the majority of database operations do not conflict
  - Does not require locking or time stamping techniques
  - Transaction is executed without restrictions until it is committed
  - Acceptable for mostly read or query database systems that require very few update transactions
  - Phases are read, validation, and write

Concurrency Control with Optimistic Methods

- Phases are read, validation, and write
  - ***Read phase*** – transaction reads the database, executes the needed computations and makes the updates to a private copy of the database values.
    - All update operations of the transaction are recorded in a temporary update file which is not accessed by the remaining transactions
  - ***Validation phase*** – transaction is validated to ensure that the changes made will not affect the integrity and consistency of the database
    - If the validation test is positive, the transaction goes to the writing phase. If negative, the transaction is restarted and the changes discarded
  - ***Writing phase*** – the changes are permanently applied to the database

Database Recovery Management

- Database recovery
  - Restores database from a given state, usually inconsistent, to a previously consistent state
  - Based on the atomic transaction property
    - All portions of the transaction must be treated as a single logical unit of work, in which all operations must be applied and completed to produce a consistent database
  - If transaction operation cannot be completed, transaction must be aborted, and any changes to the database must be rolled back (undone)

Transaction Recovery

- The database recovery process involves bringing the database to a consistent state after failure.
- Transaction recovery procedures generally make use of deferred-write and write-through techniques

Transaction Recovery

- Deferred write
  - Transaction operations do not immediately update the physical database
  - Only the transaction log is updated
  - Database is physically updated only after the transaction reaches its commit point using the transaction log information
  - If the transaction aborts before it reaches its commit point, no ROLLBACK is needed because the DB was never updated
  - A transaction that performed a COMMIT after the last checkpoint is redone using the "after" values of the transaction log

Transaction Recovery

- Write-through
  - Database is immediately updated by transaction operations during the transaction's execution, even before the transaction reaches its commit point
  - If the transaction aborts before it reaches its commit point, a ROLLBACK is done to restore the database to a consistent state
    - A transaction that committed after the last checkpoint is redone using the "after" values of the log
    - A transaction with a ROLLBACK after the last checkpoint is rolled back using the "before" values in the log

A Transaction Log for Transaction Recovery Examples

TABLE 9.13 A TRANSACTION LOG FOR TRANSACTION RECOVERY EXAMPLES

TRL ID	TRX NUM	PREV PTR	NEXT PTR	OPERATION	TABLE	ROW ID	ATTRIBUTE	BEFORE VALUE	AFTER VALUE
341	101	Null	352	START	****Start Transaction				
352	101	341	363	UPDATE	PRODUCT	54778-2T	PROD_QOH	45	43
363	101	352	365	UPDATE	CUSTOMER	10011	CUST_BALANCE	615.73	675.62
365	101	363	Null	COMMIT	**** End of Transaction				
397	106	Null	405	START	****Start Transaction				
405	106	397	415	INSERT	INVOICE	1009			1009,10016, ...
415	106	405	419	INSERT	LINE	1009,1			1009,1, 89-WRE-Q,1, ...
419	106	415	427	UPDATE	PRODUCT	89-WRE-Q	PROD_QOH	12	11
423				CHECKPOINT					
427	106	419	431	UPDATE	CUSTOMER	10016	CUST_BALANCE	0.00	277.55
431	106	427	457	INSERT	ACCT_TRANSACTION	10007			1007,18-JAN-2004, ...
457	106	431	Null	COMMIT	**** End of Transaction				
521	155	Null	525	START	****Start Transaction				
525	155	521	528	UPDATE	PRODUCT	2232/QWE	PROD_QOH	6	26
528	155	525	Null	COMMIT	**** End of Transaction				
* * * * * C *R*A* S* H * * * *									

Transaction Recovery Examples

- T101 consists of 2 UPDATE statements that reduce the QOH for product 54778-2T and increase the customer balance for customer 10011 for a credit sale of 2 units of that product
- T106 represents a credit sale of 1 unit of 89-WRE-Q to customer 10016 for $277.55 This transaction consists of 3 INSERT and 2 UPDATE statements
- T155 is an inventory update increasing QOH for 2232/QWE to 26 units
- A DB checkpoint wrote all updated database buffers to disk which is only for T101

Transaction Recovery Examples

- Last checkpoint was 423
- T101 started and finished before that checkpoint so all changes were written to disk and no action need be taken
- T106 and T155 committed after the last checkpoint so they will have their "after" values written to disk using the log